

Coursework Assignment: Idea Bookshop

22018575

School of Computing & Mathematics, Keele University

CSC-30025 Advanced Web Technologies

Dr Bappaditya Mandal

May 3rd , 2026

Word count: 2144

Table of Contents

1 Introduction.....	3
2 Requirements & Tasks	3
3 Design	5
3.1 Database.....	5
3.2 MVC	5
3.3 UI/UX	6
3.4 Ajax & Restful API	8
3.5 Security Design.....	8
3.6 Coding & Design Practices	8
4 Implementation	8
4.1 Project Structure & Routing.....	8
4.2 Database Implementation.....	9
4.3 Product Browsing & AJAX	9
4.4 Registration Implementation.....	9
4.5 Login, CAPTCHA & Sessions.....	10
4.6 Orders & Admin Order History	10
4.7 Security Implementation.....	11
4.8 SEO & Responsive Design	11
5 Testing & Validation	11
5.1 Overview	11
5.2 SQL Injection Testing	12
5.3 Cross-Site Scripting (XSS) Testing.....	12
5.4 Session Management Testing	12
5.5 CAPTCHA Testing	13
5.6 Lighthouse.....	13
6.0 Conclusion	13

1 Introduction

The Idea Bookstore is a multi-tier web application developed for CSC-30025 Advanced Web Technologies. Its purpose is to allow customers to browse books, register, log in, order products and view order history, while managers can retrieve customer order data through search, filtering and REST-style JSON endpoints. The system demonstrates industry adaptation of web technologies by combining PHP, MySQL, JavaScript, React, AJAX, CSS and MVC into one application.

This Application separates presentation (views), logic (controllers), and data (models). This approach reflects best practice in PHP development, where applications are divided into self-contained components to improve maintainability and scalability (Nixon, 2025, p. 91). This system includes a React-based frontend with live validation and RESTful API endpoints for administrative access. Security mechanisms include input validation, prepared statements, session management, and CAPTCHA. The overall system structure is illustrated in Figures 1 and 2.

2 Requirements & Tasks

Task	Comments
1. Bookstore company webpage	Implemented an MVC-based bookstore with navigation, category browsing, product cards, images, and AJAX product loading without full page reloads.
2. SQL schema	Created Accounts, products, orders, and CaptchaImages tables with primary keys, foreign keys, unique email constraint, and normalised order storage (Nixon, 2025, pp. 209-222).
3. Registration Page	Built React live validation for username, phone, email, password, and confirm password. Email availability is checked asynchronously; final validation is repeated server-side. Issue: full React dependency installation was restricted on the university server, so React was used through browser-compatible scripts.

4. Login Page	Implemented registered-user login with CAPTCHA, password_verify(), sessions, protected routes, and user feedback.	
5. Security of Registration & Login	Uses prepared statements, password hashing, CAPTCHA, server-side validation, session checks, and output escaping to prevent SQL injection and XSS	
6. SEO	Added page titles, descriptions, keywords, author metadata, robots metadata, Open Graph tags, alt text, sitemap, robots.txt, and security.txt.	
7. Registration with React	The registration form uses React to implement live keystroke validation, while email availability is checked asynchronously before submission, similar to Nixon’s Gmail example (Nixon, 2025, p. 15).	
8. Data Management	A) Customer registration table stores user data with a unique email and Hashed passwords.	
	B) The orders table stores purchases using foreign keys (account_id, product_id), linking users and products without duplicating customer or product data.	
9. RESTful webservice	REST endpoint retrieves orders by ID and filters parameters, returning JSON responses. Implementation detailed in Section 4.6.	
10. Additional information	MVC structure ensures modularity and maintainability. Consistent naming conventions (CamelCase, PascalCase, snake_case) and modular CSS improve code quality.	

3 Design

3.1 Database

The database uses four relational tables: Accounts, products, orders, and CaptchaImages. This reflects relational database design because data is separated into logical entities and linked using keys (Nixon, 2025, pp. 209-222). The schema is normalised to reduce redundancy: user details are stored in Accounts, book details in products, and purchases in orders using account_id and product_id foreign keys. This creates a one-to-many relationship between users and orders and avoids duplicating customer or product data. Implementation is detailed in Section 4.2, and the ER diagram is shown in Figure 1.

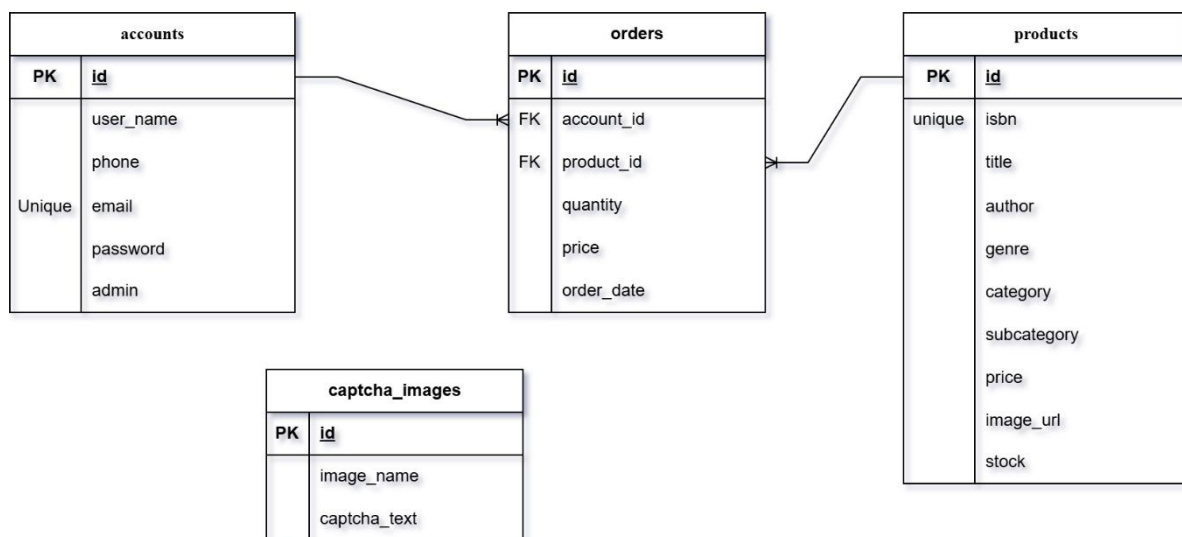


Figure 1. Crow's Foot Diagram for Idea bookstore

3.2 MVC

The system uses MVC, separating views, controllers and models. This supports modularity because Nixon argues that complex PHP websites become difficult to maintain as one monolithic script and should instead be divided into self-contained components (Nixon, 2025, p. 91). Therefore, the project was refactored into specialised controllers and models, including AuthenticationController, BookController, AdminController, and UserAccountsModel, aligning with MVC principles (Pop and Altar, 2014).

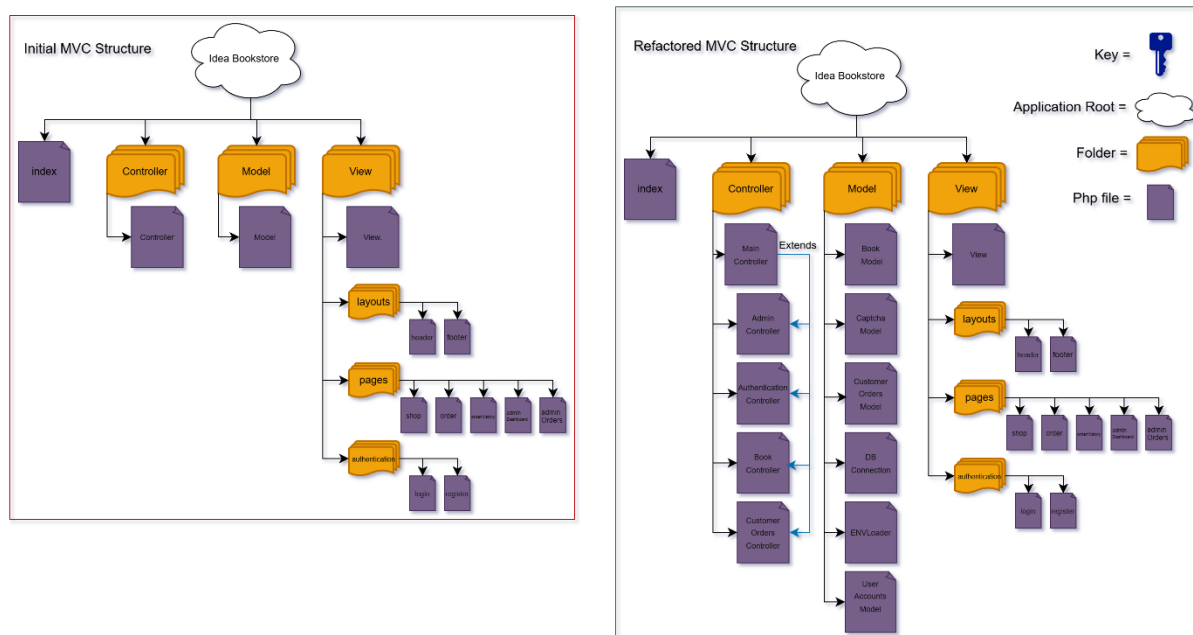


Figure 2. Original & Refactored MVC architecture

3.3 UI/UX

The UI/UX design was influenced by Waterstones, World of Books, and AwesomeBooks, specifically their navigation structure, hover-based category dropdowns, and use of “Account” instead of separate login/register links. These were implemented in the Idea webpage. Design progression is shown in Figure 4, with key inspired features highlighted in Figure 3.

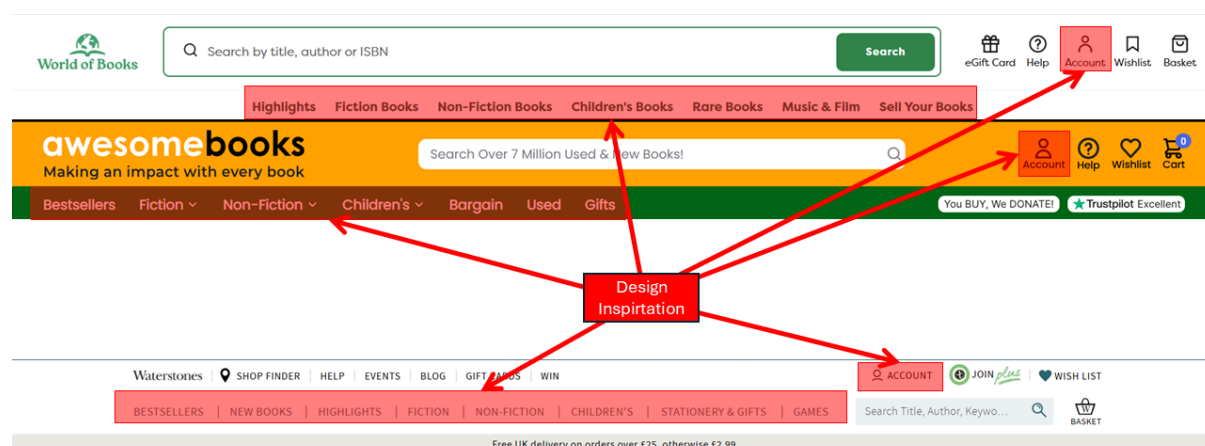


Figure 3



Figure 4

The coursework requires a two-level dropdown system (Kids/Adults → subcategories). This was implemented in the admin filtering system for scalability, while the main shop uses simplified category navigation to reduce complexity. (see Figure 5).

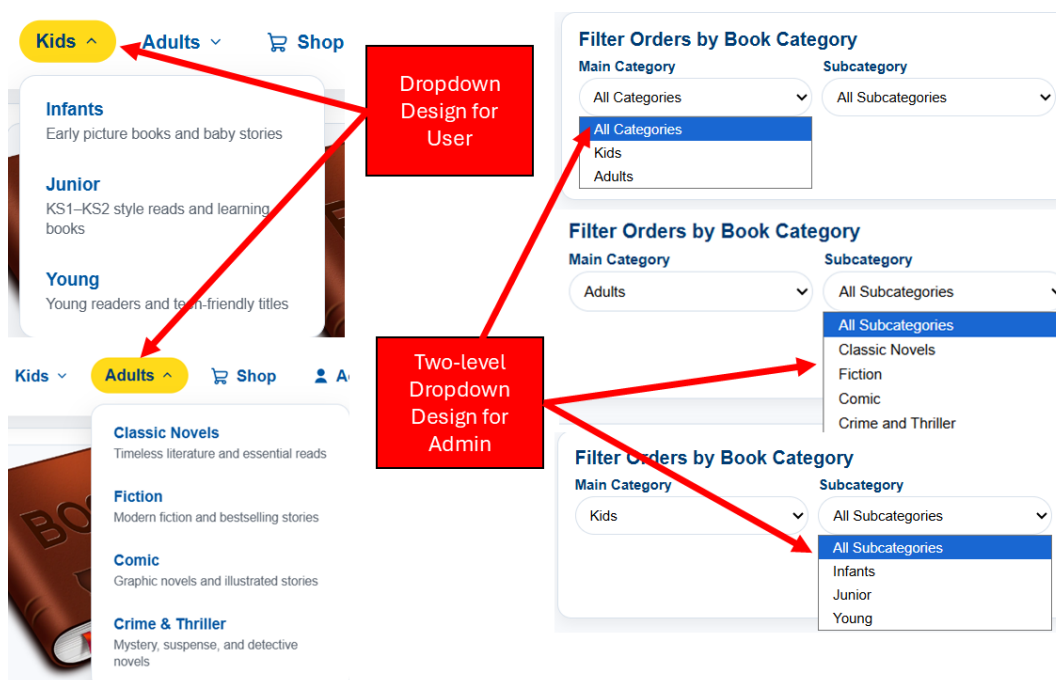
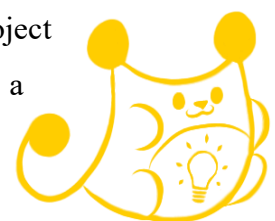


Figure 5: Dropdown Design.

The system uses the “Idea Company” branding from a previous project *Database Systems - Idea Furniture Company* (22018575, 2024), using a blue/yellow colour scheme and a custom mascot (“Meowchie”) to maintain a brand identity.



The layout uses responsive grid/card components and centred forms to support usability across devices, informed by online bookstore and login-form design tutorials (Mr. Web Designer, 2021; Ludiflex, 2025). React live validation, clear input structure and feedback messages improve form usability by reducing user error before submission (Nixon, 2025, pp. 267-280).

3.4 Ajax & Restful API

AJAX is used so product data can be requested in the background without reloading the full page. This improves responsiveness because the browser and server exchange data while the page remains active (Nixon, 2025, pp. 403-413). The system returns JSON from PHP controllers, allowing JavaScript to update book cards dynamically. REST-style endpoints use URL parameters and return JSON.

3.5 Security Design

The security design combines server-side validation, prepared statements, password hashing, output escaping, CAPTCHA, and session-based authentication. Form input is treated as untrusted because browser validation and JavaScript checks can be bypassed (Nixon, 2025, pp. 375-392). Prepared statements protect SQL queries, while escaping functions reduce XSS risk. Session-based authentication stores login state server-side rather than storing credentials in cookies (Nixon, 2025, pp. 259-265; Nixon, 2025, pp.289-309).

3.6 Coding & Design Practices

Consistent coding standards are used, including naming conventions CamelCase/PascalCase for application code and snake_case for database fields. Modular CSS separation comments, JSDoc-style comments, and self-commenting code, improving readability, and supporting future enhancement.

4 Implementation

4.1 Project Structure & Routing

The website starts at `index.php?action=shop`. `index.php` acts as the front controller: it loads `Bootstrap.php`, reads the action parameter, and routes each request to the corresponding controller method. For example, product AJAX requests are routed to `BookController::fetchBooks()`, while manager order API requests are routed to `AdminController::fetchOrderById()`.

```
$action = $_GET['action'] ?? 'shop';  
'fetchBooks' => fn() => $bookController->fetchBooks(),  
'apiOrder' => fn() => $adminController->fetchOrderById()
```

`Bootstrap.php` centralises dependency creation by instantiating models and injecting them into controllers, reducing repeated setup code and supporting modularity.

```
'book' => new BookController($bookModel),
```



```
'admin' => new AdminController($orderModel)
```

4.2 Database Implementation

Database access is handled in DB_Connection.php. Credentials are loaded from .env through EnvLoader.php, separating configuration from source code.

```
EnvLoader::load(__DIR__ . '/.env');  
self::$connection = mysqli_connect($host, $username,  
$password, $databaseName);
```

The Model classes perform database operations using prepared statements. For example, BookModel optionally filters books by category/subcategory using bound parameters. This separates SQL structure from submitted values, reducing SQL injection risk (Nixon, 2025, pp. 259-265).

```
$sql .= " AND category = ?";  
$stmt->bind_param($types, ...$params);
```

4.3 Product Browsing & AJAX

The shop page uses shop.js to request product data asynchronously. When a category or subcategory is selected, JavaScript builds a request URL for index.php?action=fetchBooks.

```
let url = "index.php?action=fetchBooks";  
url += "&category=" + encodeURIComponent(category);
```

BookController::fetchBooks() retrieves matching products from BookModel::getBooks() and returns JSON. The frontend then renders book cards dynamically without a full page reload, reflecting modern JSON-based asynchronous communication (Nixon, 2025, pp. 403-413).

```
$books = $this->bookModel->getBooks($category, $subcategory);  
$this->jsonResponse(['success' => true, 'data' => $books]);
```

4.4 Registration Implementation

Registration is implemented with a React component in register.js. It stores form values in component state and validates username, phone, email, password, and confirmation fields on each keystroke. (Nixon, 2025, pp. 483-506).

```
class RegistrationForm extends React.Component
```

Email availability is checked asynchronously using fetch(), allowing feedback without leaving the registration page (Nixon, 2025, pp. 403-413).

```
fetch("index.php?action=checkEmailExists&email=" +  
encodeURIComponent(email.trim()))
```

The form follows a standard form-handling lifecycle: input is submitted, validated in PHP, and inserted only if valid. Server-side validation is repeated because client-side checks can be bypassed (Nixon, 2025, p. 375). Passwords are stored as hashes rather than plain text (Nixon, 2025, pp 296-298).

```
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);
```

4.5 Login, CAPTCHA & Sessions

The login page loads a random CAPTCHA from the database and stores its ID/image in the session.

```
$_SESSION['loginCaptchaId'] = $captcha['id'];  
$_SESSION['loginCaptchaImage'] = $captcha['imageName'];
```

During login, the controller validates email, password, and CAPTCHA. `password_verify()` checks the submitted password against the stored hash without comparing plain-text passwords (Nixon, 2025, pp. 296-298).

```
$this->captchaModel->captchaMatches($captchaId,  
$captchaInput); password_verify($password, $user['password']);
```

After successful login, the session ID is regenerated and session variables store authentication state and user details. Regenerating the session ID after successful login helps reduce session fixation risk because the authenticated session receives a new identifier after the user's authentication state changes (Nixon, 2025, pp 302-208).

```
session_regenerate_id(true);  
$_SESSION['isLoggedIn'] = true;  
$_SESSION['userId'] = (int) $user['id'];
```

Session variables stored in `$_SESSION` allow the system to recognise authenticated users across protected pages without storing sensitive login credentials in client-side cookies.

4.6 Orders & Admin Order History

Users can browse products without logging in, but ordering requires authentication. The order controller checks login status, blocks admin ordering, validates the product ID, checks stock, and inserts the order.

```
$this->requireLogin();  
$this->blockAdminOrdering();  
$this->orderModel->createOrder($userId, $productId,  
$product['price']);
```

Orders are stored using `account_id` and `product_id` foreign keys, implementing the one-to-many user/order relationship described in Section 3.1 (Nixon, 2025, pp. 201-204).

```
INSERT INTO orders (account_id, product_id, quantity, price)
VALUES (?, ?, ?, ?)
```

Administrators can retrieve order data through REST-style URLs such as:

```
index.php?action=apiOrder&id=3
```

This endpoint returns JSON, allowing order data to be retrieved through URL parameters.

4.7 Security Implementation

Security is implemented through prepared statements, server-side validation, sessions, CAPTCHA, password hashing, and output escaping. Prepared statements prevent SQL injection by ensuring user input is never interpreted as executable SQL (Nixon, 2025, pp. 259-263).

```
$stmt = $this->connection->prepare($sql);
$stmt->bind_param("s", $email);
```

XSS is reduced by escaping output before rendering database or user content. `htmlspecialchars()` / `htmlspecialchars()` convert dangerous HTML characters into safe text (Nixon, 2025, pp. 276-279)

```
echo htmlentities($order['title'], ENT_QUOTES, 'UTF-8');
```

Protected routes use `requireLogin()`, while admin-only routes use `requireAdmin()`. These are tested in Section 5.

4.8 SEO & Responsive Design

SEO is implemented in `header.php` using page titles, descriptions, keywords, author metadata, robots metadata, and Open Graph tags..

```
<meta name="description" content="Idea Bookstore is an online
bookstore...">
<meta property="og:title" content="Idea Bookstore">
```

Images include alt text. The project includes a `sitemap.xml`, `robots.txt`, and `security.txt`. Responsive design is handled through modular CSS, a mobile navigation toggle, and grid/card layouts.

5 Testing & Validation

5.1 Overview

The system was tested for functionality, security, robustness, and performance. Testing focused on SQL injection, XSS, session enforcement, CAPTCHA behaviour, and Lighthouse results.

This validates the security measures described in Section 3.5 and implemented in Sections 4.5-4.7.

5.2 SQL Injection Testing

SQL injection was tested on login, registration, AJAX endpoints, and URL parameters. Basic payloads were rejected by validation:

```
' OR '1'='1
```

A valid-format malicious input was also tested to check backend protection:.

```
Aa1!' OR '1'='1
```

These inputs passed validation and did not bypass authentication. The system returned “*Incorrect email or password*”, confirming that prepared statements prevented SQL logic from being altered.

5.3 Cross-Site Scripting (XSS) Testing

XSS was tested through input fields and URL parameters. Basic payloads were rejected by username validation:

```
<script>alert('XSS')</script>  
<img src=x onerror=alert(1)>
```

The admin search also accepts only numeric input, reducing script injection risk. Advanced URL-based payloads were then tested:

```
"><img src=x onerror=alert(1)>
```

and double-encoded payloads:

```
%253Cscript%253Ealert(1)%253C/script%253E
```

No scripts executed, and the application rendered normally. This confirms that input was not interpreted as executable HTML and that output escaping was effective, consistent with the use of `htmlspecialchars()` (Nixon, 2025, p. 283).

5.4 Session Management Testing

Protected routes were tested without authentication:

```
index.php?action=orderHistory
```

The system returned “*Please login first*”, confirming that `requireLogin()` is enforced. After logout, using the browser back button did not restore access, showing that authenticated session state was removed correctly. This confirms secure session lifecycle management.

5.5 CAPTCHA Testing

CAPTCHA functionality was tested as part of the login process to prevent automated access,

- Submitting the login form with an empty CAPTCHA field resulted in login failure
- Entering an incorrect CAPTCHA prevented authentication
- Entering the correct CAPTCHA allowed successful login with valid credentials

5.6 Lighthouse

Performance testing using Google Lighthouse achieved a score of 100 across Performance, Accessibility, Best Practices, and SEO (see Figure 6). Key metrics such as low First Contentful Paint, zero Total Blocking Time, and no layout shift indicate a fast, stable, and responsive application.

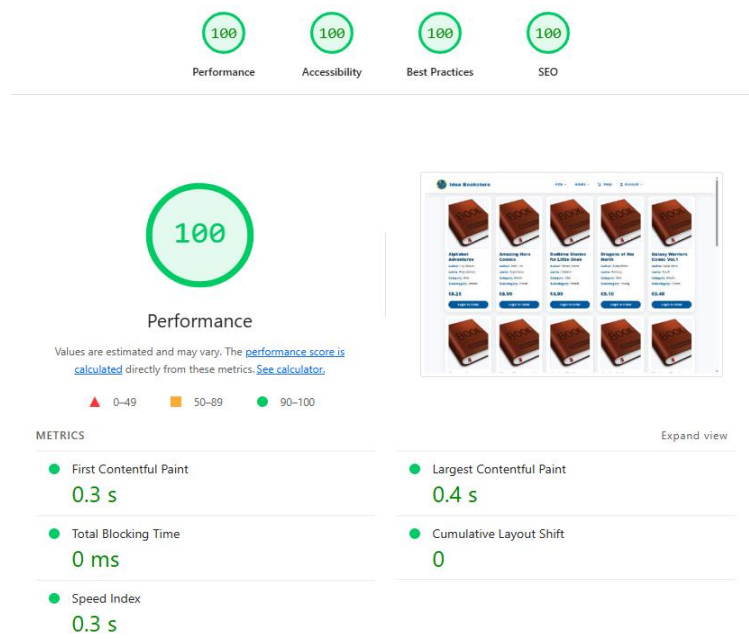


Figure 6: Lighthouse Scores

6.0 Conclusion

The Idea Bookstore meets the coursework requirements by delivering a secure multi-tier web application using PHP, MySQL, JavaScript, React, AJAX and MVC. Users can browse books, register with live validation, login using CAPTCHA, place orders and view order history, while administrators can retrieve customer order data through filtering and REST-style JSON endpoints.

The system demonstrates industry adaptation of web technologies through modular MVC architecture, asynchronous JSON-based interaction and relational data management. Its design applies modular PHP principles, normalised relational tables, primary and foreign keys, prepared statements, password hashing, output escaping and session management to support maintainability, scalability and security.

Future improvements could include an expanding the admin interface for adding books, updating stock, editing product details and managing catalogue availability, alongside live stock reduction, payment integration, email confirmation, compiled React using Node.js, and catalogue features such as new arrivals or featured books.

Use of Artificial Intelligence (AI)

I acknowledge the use of assistive AI tools, ChatGPT, for spelling, grammar, punctuation and sentence-structure checking in this report.

References

22018575 (2024) Database Systems Coursework Assignment – Idea Furniture Company. CSC-20002 Database Systems, School of Computing and Mathematics, Keele University. Unpublished coursework.

AwesomeBooks (2025) Cheap New & Used Books Online with Free Delivery | Awesome Books. Available at: <https://www.awesomebooks.com/>.

Ludiflex (2025) Login Form in HTML CSS & JavaScript, YouTube. Available at: https://www.youtube.com/watch?v=UYVJ_ji1Jfk (Accessed: 16 April 2026).

Mr. Web Designer (2021) Create A Responsive Online Book Store Website Design Using HTML - CSS - JavaScript || Step By Step, YouTube. Available at: <https://www.youtube.com/watch?v=6XmDdIRmg84> (Accessed: 20 April 2026).

Nixon, R. (2025) Learning PHP, MySQL & JavaScript: a step-by-step guide to creating dynamic websites. 7th edn. Sebastopol, CA: O'Reilly Media, Inc.

Pop, D.P. and Altar, A. (2014) 'Designing an MVC Model for Rapid Web Application Development', *Procedia Engineering*, 69, pp. 1172–1179.

Waterstones (2025) Buy books, stationery and gifts, online and in store | Waterstones. Available at: <https://www.waterstones.com/>.

World of Books (2017) World of Books | Buy cheap second-hand books online. Available at: <https://www.worldofbooks.com/>.